Informatics II, Spring 2026, Exercise 2

Publication of exercise: March 2, 2026
Publication of solution: March 9, 2026
Exercise classes: March 9 - 13, 2026

Task 1 (Easy)

1.1 Analyse the recursive algorithm `RecursiveAlgo` in the program `WhatDoIDo`.

Explain in your own words what the algorithm does and what it returns when executed. Also identify the important building blocks of the recursion:

- the base case(s)/terminating condition(s),
- the recursive step,
- how the algorithm makes progress and why it eventually terminates.

1.2 What is the time complexity of the algorithm. Assume that the input has length n.

Program: WhatDoIdo

```
RecursiveAlgo(str, s, e){
  if s==e then
  ∟ return true
  if str[s] != str[e] then
  ∟ return false
  if s < e + 1 then
  ∟ return RecursiveAlgo(str, s + 1, e - 1)
  return true
}

main() {
  str = "racecar"
  n = strlen(str)
  res = false
  if n > 0 then
  ∟  res = RecursiveAlgo(str, 0, n - 1)
  printf("%s", res?"true":"false")
}
```

Task 2 (Easy)

Analyse the function `printRec(int n)` and answer the following questions.

```
void printRec(int n)
{
  if (n == 0)
    return;

  printf("%d", n%2);

  printRec(n/2);
}
```

2.1 What is the output of the function call `printRec(20)`?

2.2 Write down the sequence of recursive function calls for the call `printRec(20)`.

For each call, state:

- the argument value with which the function is called,
- the calculation performed in that call,
- and what is printed before the next recursive call.

2.3 To get the output `10100` for $n = 20$ using the function `printRec(n)`, which changes would you make in the function `printRec(n)`?

Hint: The desired output is the reverse of the original output.

Task 3 (Medium)

The Tower of Hanoi is a puzzle with three pegs (A, B, C) and N disks of different sizes. Initially, all disks are stacked on peg A with the smallest disk on top and the largest at the bottom. Your goal is to move the entire stack from A to B.

You may only follow these rules:

- In one move, you can take *only the top disk* from a peg and place it onto the top of another peg.
- You may never place a *larger* disk on top of a *smaller* disk.

The picture below shows the state of a game with 7 disks after 4 moves.

```
Tower of Hanoi pegs after 4 moves:



    *******
   *********
  ***********                        *
 *************       *****          ***
```

3.1 Before implementing anything, think about the strategy:

- Moving **1** disk from one peg to another is straightforward, you just move it.
- How would you move **2** disks from A to B (using C), while respecting the rules?
- Now generalize this idea: If you want to move n disks from A to B (using C), how can you break the problem into smaller subproblems of the same type?

Explain the *recursive idea* in your own words (base case? what has to happen before/after moving the largest disk?). Do not write C code here, focus on the reasoning/strategy.

3.2 Use the file `hanoi_skeleton.c` from OLAT and your recursive idea from subtask 3.1 to complete the missing function `Hanoi(...)`, so that the program correctly solves the Tower of Hanoi puzzle. It shall be possible to choose the number of disks (up to 9) and to display the stacks in a terminal after a given number of moves. Finally, display the stacks for **8 disks after 213 moves**.

In the provided code, each peg is stored as an integer whose digits represent the disks on that peg, where the last digit corresponds to the top disk. The value of a digit also represents the size of the disk: `1` is the smallest disk, `2` the next larger one, and so on.

For example, if A is `4321`, then the top disk is `1`.

To move the top disk from A to B, you can use the following steps:

`disk = A % 10` (take top disk)

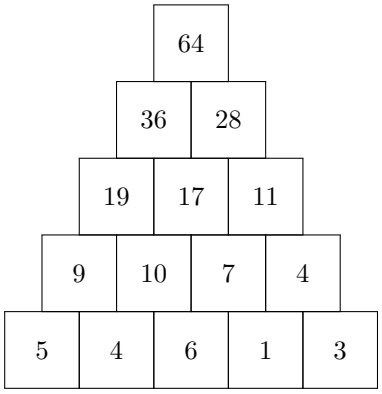
`B = B * 10 + disk` (push onto B)

`A = A / 10` (remove from A)

Example: If `A = 4321` and `B = 0`, after one move you get `A = 432` and `B = 1`.

Task 4 (Medium)

Write a recursive function that takes in an integer array of length n and recursively reduces the array by adding two neighbouring numbers. Each new array is stacked on top of the previous array. Thus, the output is a pyramid in which each element i is the sum of the left and right child of the element i in the level below as depicted below for the case $A = [5, 4, 6, 1, 3]$.



Hints:

- Do it like in Task 3: first think about the recursive idea (base case + recursive step) on paper, then code.
- Reminder: You can create a temporary array of a fixed length in C like `int t[length];` (e.g. `int t[length-1];`).
- Use the file `triangle_sum_skeleton.c` from OLAT and complete the missing recursive function. Use the provided print function to print one level after each recursive call.